

1. What is Encapsulation?

Encapsulation is the practice of bundling data (properties) and actions (methods) into a single unit (a class) while **restricting direct access** to the inner workings.

Think of it like a medical capsule: the medicine (data) is hidden inside, and you only interact with the outer shell (public methods).

2. Private Properties

In Python, you can signal that a property is **private** by using a double underscore `__` prefix. This prevents direct external access.

```
class Account:
    def __init__(self, balance):
        self.__balance = balance # Private

acc = Account(100)
# print(acc.__balance) # Raises AttributeError
```

3. Accessing Data: Getters

Since private data cannot be reached directly, we use **Getter methods** to safely retrieve personal or sensitive values.

```
class Person:
    def __init__(self, age):
        self.__age = age

    def get_age(self):
        return self.__age
```

This allows the class to control exactly how and when data is viewed.

4. Modifying Data: Setters

Setter methods allow us to change private data while performing **validation** to ensure the data is correct before saving it.

```
def set_age(self, age):  
    if age > 0:  
        self.__age = age  
    else:  
        print("Invalid age!")
```

This prevents bugs where an object might end up with impossible data (like a negative age).

5. Protected Properties

Python uses a single underscore `_` as a convention for **Protected** data. It doesn't physically block access, but it's a "gentleman's agreement."

Convention: A single `_` tells other developers: "I am intended for internal use only. Modify me at your own risk."

6. Private Methods

Just like properties, methods can be made private using `__`. These are typically "helper" functions used for internal logic that the user doesn't need to see.

```
class Calculator:
    def __validate(self, n): # Private
        return isinstance(n, int)

    def add(self, n):
        if self.__validate(n): # Call internally
            self.val += n
```

7. Name Mangling

When you use `__`, Python performs **Name Mangling**. It renames the variable internally to `__ClassName__variable`.

```
Internals of Person Class  
print(p1._Person__age) # Accessing mangled name
```

While possible, accessing mangled names is considered extremely poor practice.

8. The Benefits of Security

1. **Data Protection:** No accidental overrides from external scripts.
2. **Validation:** Guarantee logic (e.g., student grades between 0-100).
3. **Decoupling:** Change internal logic without breaking the app.

9. Practical Exercise

Knowledge Check

Question: How do you make a property private in Python?

☐ Use a single underscore prefix: `_prop`

☒ Use a double underscore prefix: `__prop`

☐ Use the `private` keyword

10. Mastery Summary

- ✓ **Encapsulation** shields internal state.
- ✓ `__private` is enforced by Python.
- ✓ `_protected` is a naming convention.
- ✓ Use **Getters/Setters** for safety.
- ✓ **Name Mangling** is the secret behind private security.

codehive

Security & Encapsulation Series